

Agenda

1. Log Basics
2. Importance Of Constraints
3. Space Complexity
4. Introduction to Array
5. Reverse an array
6. Rotate an array
7. Dynamic Arrays

LOG

logarithm is inverse of power/exponents.

$$\log_b a \quad \text{log } a \text{ to the base } b$$

↓
what should be power of b , so that we get a

$$b^? = a$$

$$\log_2 64 = 6$$

$$2^? = 64$$

$$\log_3 27 = 3$$

$$3^? = 27$$

$$\log_5 125 = 3$$

$$5^? = 125$$

$$\log_2 10 = 3$$

$$2^? = 10$$

$$2^{3.xx} = 10$$

$$\begin{array}{ccc} 2^3 & 2^{3.} & 2^4 \\ \downarrow & \vdots & \downarrow \\ 8 & 10 & 16 \end{array}$$

$$\log_2 8 = 3$$

$$\log_a a = 1$$

$$\log_2 N$$

Base is 2

$$\text{If } 2^k = N$$

$$\log_2 N = k$$

$$2^k = N$$

Take $\log_2$ on both sides

$$\log_2 (2^k) = \log_2 N$$

$$\Rightarrow k = \log_2 N$$

Iterations (Less)



Big O () : TC

$$\text{Iterations} \rightarrow 4N^2 + 3N + 1 \quad N^0$$

↓
 $4N^2$

↓
 N^2

TC: $O(N^2)$

Time Limit $\rightarrow 1$ sec

Servers can run $10^7 - 10^8$ iterations in 1 sec

TLE (Time Limit Exceeded)

$$10^3 = 1000$$

$$2^{10} \approx 10^3$$

$$2^{10} = 1024$$

$$\log_2 2^{10} = 10$$

$$\log_2 10^3 = 10$$

Constraints (size of Input)

① $1 \leq N \leq 10^5$

Complexity	Iterations (Max)	Works?
$O(N^3)$	$(10^5)^3 = 10^{15}$	X TLE
$O(N^2 \log N)$	$(10^5)^2 \times \log_2 10^5$ $10^{10} \times 15. -$	X TLE
$O(N^2)$	$(10^5)^2 = 10^{10}$	X TLE
$O(N \log N)$	$10^5 \times \log_2 10^5$ $10^5 \times 15. -$	✓

② $1 \leq N \leq 10^6$

Complexity	Iterations	Works?
$O(N^3)$	$(10^6)^3 = 10^{18}$	X TLE
$O(N^2 \log N)$	$(10^6)^2 \log_2 10^6$ $10^{12} \times -$	X TLE
$O(N^2)$	$(10^6)^2 = 10^{12}$	X TLE
$O(N \log N)$	$10^6 \times \log_2 10^6$ $10^6 \times 20 \approx 10^7 \times 2$	$10^7 - 10^8$ Maybe
$O(N)$	10^6	✓

$$2^{20} \approx 10^6$$

- ③ $1 \leq N \leq 100$ N^3 will also work
- ④ $1 \leq N \leq 20$ 2^N will also work

- ① Read the Question and constraints
↓
I/P, process, O/P
- ② Formulate an idea or logic
- ③ Verify correctness of logic
- ③ Mentally develop a rough idea of loops and pseudocode
- ④ Determine TC based on pseudocode
is TC feasible as per constraints?
It won't result in TLE (at max $10^7 - 10^8$ iterations)
- ⑤ Re-evaluate the idea if TLE is met, otherwise proceed with code.

Space Complexity $\rightarrow$ Big O Notation



$O()$

max space taken by your algo
at any point of time

Auxiliary space /

Extra space

Total Space = input space + space taken by algo + output space

func (int n) $\rightarrow$ 4 B

int	long
$\downarrow$	$\downarrow$
4 B	8 B

	int x $\rightarrow$ 4 B	}	Space taken by your algo
	int y $\rightarrow$ 4 B		
	long z $\rightarrow$ 8 B		

Total program space = 4 + 4 + 4 + 8
= 20 B

SC $\rightarrow$ 16 B
 $O(1)$

1. func (int N) {

int arr1[10] $\rightarrow 4 \times 10 = 40$ B

int x $\rightarrow 4$ B

int y $\rightarrow 4$ B

long z $\rightarrow 8$ B

int arr2[N] $\rightarrow 4 \times N = 4N$ B

}

$$SC: 40 + 4 + 4 + 8 + 4N \\ = (56 + 4N) \text{ Bytes}$$

$O(N)$

2. func (int N) {

int x $\rightarrow 4$ B

int y $\rightarrow 4$ B

long z $\rightarrow 8$ B

int arr[N] $\rightarrow 4N$ B

long l[N][N] $\rightarrow 8 \times N^2 \rightarrow 8N^2$ B

}

$$SC: 4 + 4 + 4 + 4N + 8N^2 \\ \Rightarrow (12 + 4N + 8N^2) \text{ B}$$

$O(N^2)$

```

3. int maxArr (int arr[], int N) {
    int ans = arr[0]
    for (i from 1 to N-1) {
        ans = max(ans, arr[i])
    }
    return ans
}

```

Total program space = I/P + Space for your algo + O/P
 $\downarrow \qquad \qquad \qquad \downarrow \qquad \qquad \qquad \downarrow$
 $4N + 4 \qquad \qquad \qquad 4 \text{ B} \qquad \qquad \qquad 4$
 $SC: O(1)$

```

fn (int arr[], int N) {

```

```

    int ans = arr[0]
    for (i from 1 to N-1) {
        int x[N]
        |||
    }
    return ans
}

```

$i \rightarrow 4 \text{ B}$
 $x \rightarrow 4N \text{ B}$

$SC: 4 + 4N$
 $O(N)$



Intro to arrays

collection of same type of data

① `int age-apurva, age-sakshi,`

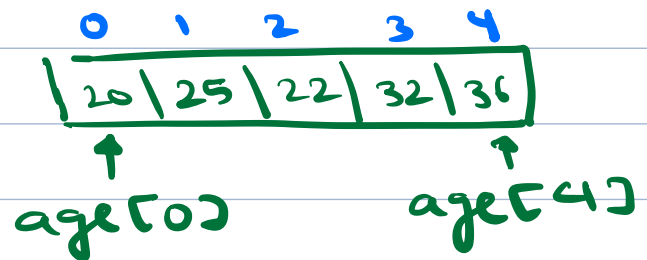
② `int age[100]`

↓ data type ↓ name of array ↓ size of array

↓
e.g. `int, float, char` etc.
(any data type)

③ Access elements → index

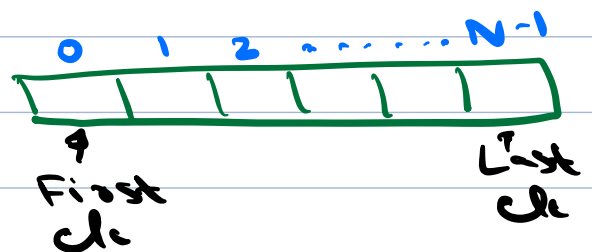
`int age[5]`



Access element at index $i \rightarrow arr[i]$

`int arr[N]`

idx $\rightarrow [0 \dots N-1]$



Time complexity to access i^{th} index
in array of size N

`arr[i]  $\rightarrow O(1)$`

$1 \text{ iter} \rightarrow O(1)$
 $N \text{ iter} \rightarrow$
 $N \times O(1)$
 $= O(N)$

```

fn printArray ( arr [], n ) < N iter >
    for ( i = 0 ; i < N ; i++ ) <
        print ( arr[i] )
    >
  
```

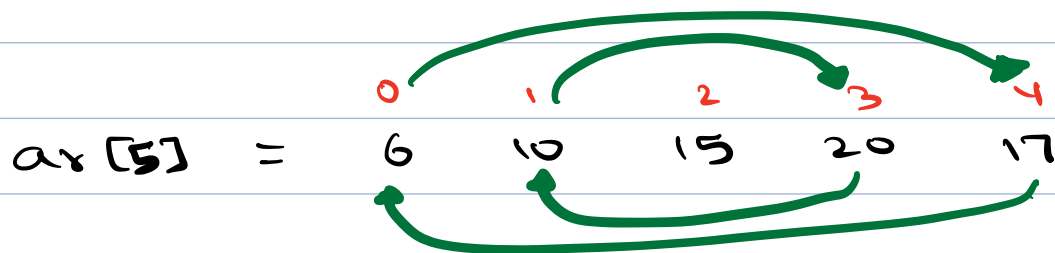
$TC : O(N)$
 $SC : O(1)$

$\text{int arr}[5] = \langle \overset{0}{5}, \overset{1}{-4}, \overset{2}{8}, \overset{3}{9}, \overset{4}{10} \rangle$
 1st 5th

$\text{print (arr[0] + arr[4])}$

10:42

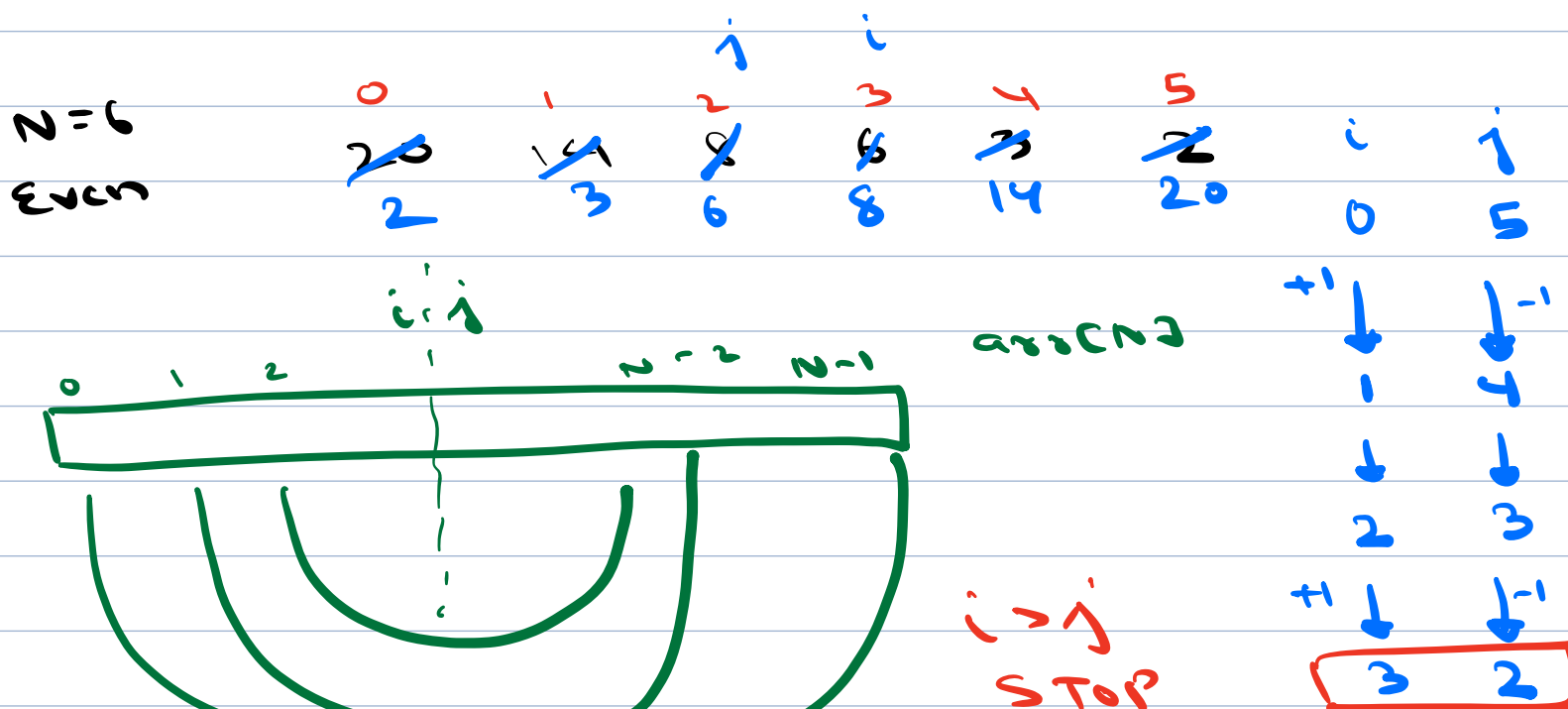
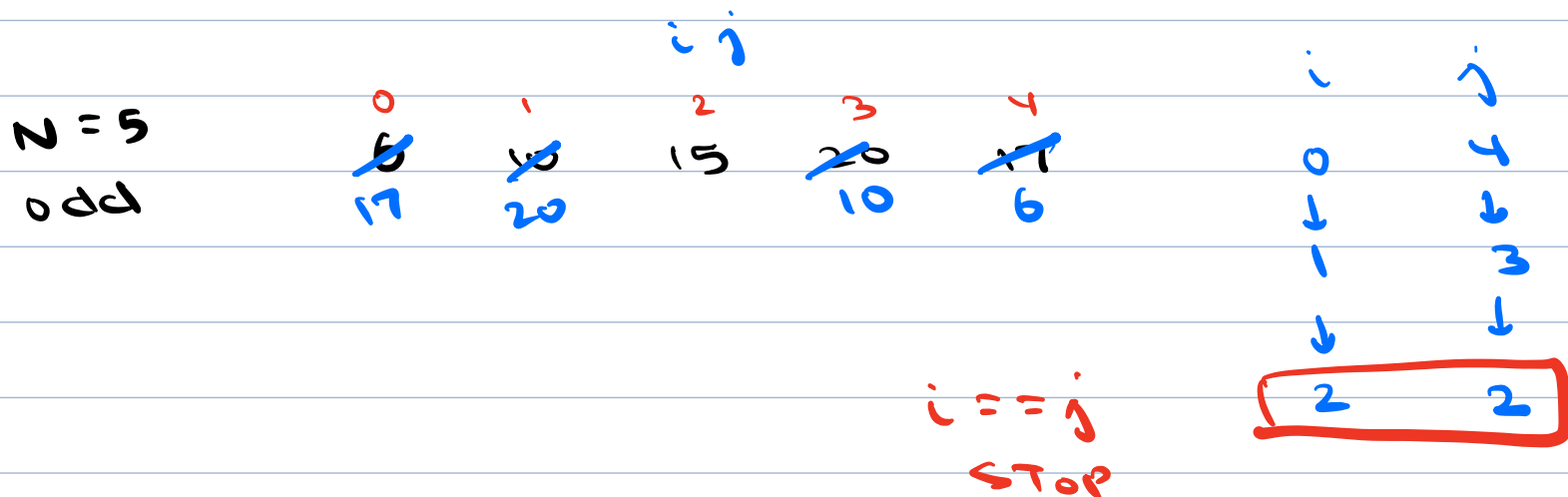
1. Given an arr of size N , reverse it



O/P =

0	1	2	3	4
17	20	15	10	6

Approach: Swap elements



void revArray (int ar[], int N) {

int i = 0, j = N - 1

while (i < j) {

int temp = ar[i]

ar[i] = ar[j]

ar[j] = temp

i++

j--

}

}

$i \geq j$
STOP

temp 20

ar[i] a 20 30

ar[j] b 30 20

T.C: $O(N)$

S.C: $O(1)$

2 dc $\rightarrow$ 1 itr

1 dc $\rightarrow \frac{1}{2}$ itr

N dc $\rightarrow \frac{N}{2}$ itr

2. Given an arr, reverse part of arr

$\downarrow$
 $l - r$

$N = 5$

$\begin{matrix} & 0 & 1 & 2 & 3 & 4 \\ & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ \langle 1, & 2, & 3, & 4, & 5 \rangle \end{matrix}$

$l = 1$

$r = 3$

$\begin{matrix} & \downarrow & & \downarrow & \\ \langle 1, & 4, & 3, & 2, & 5 \rangle \end{matrix}$

void revPartArray (int arr[], int N, int l, int r) {

int i = l, j = r

while (i < j) {

int temp = arr[i]

arr[i] = arr[j]

arr[j] = temp

i++ j--

}

TC: $O(N)$

SC: $O(1)$



arr[N]

No. of elements in $[l, r] = r - l + 1$

2 ele $\rightarrow$ 1 itr

1 ele $\rightarrow \frac{1}{2}$ itr

$(r - l + 1)$ ele $\rightarrow \frac{r - l + 1}{2}$ itr

$l \rightarrow 0 \quad r \rightarrow N-1$
worst case

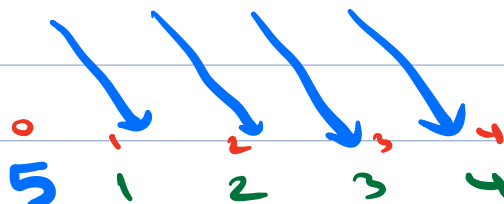
3. Given an array of size N , rotate the array from $R \rightarrow L$ K times

$N = 5$

arr = $\langle 1, 2, 3, 4, 5 \rangle$

$K = 2$

$K = 1$



$K = 2$

$\langle 4, 5, 1, 2, 3 \rangle$

arr = $\langle 1, 2, 3, 4, 5, 6 \rangle$

$K = 4$

O/P $\rightarrow \langle 3, 4, 5, 6, 1, 2 \rangle$

$K = 1$

$\langle 1, 2, 3, 4, 5, 6 \rangle$
 $\langle 6, 1, 2, 3, 4, 5 \rangle$

$K = 2$

$\langle 5, 6, 1, 2, 3, 4 \rangle$

$K = 3$

$\langle 4, 5, 6, 1, 2, 3 \rangle$

$K = 4$

$\langle 3, 4, 5, 6, 1, 2 \rangle$

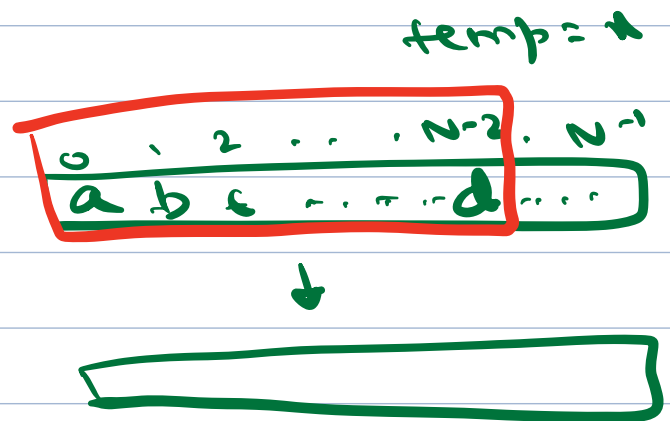
Approach 1: Brute Force

Rotate array 1 time $\rightarrow$ K times

① $temp = arr[N-1]$

② shift ele from index $(N-2$ to $0)$ by 1 index

③ $arr[0] = temp$



function rotate (int arr[], int N, int k) {

for (cnt = 1; cnt $\leq$ k; cnt++) {
 int temp = arr[N-1]

 for (i = N-2; i $\geq$ 0; i--) {

 arr[i+1] = arr[i]

 arr[0] = temp

TC: $O(NK)$

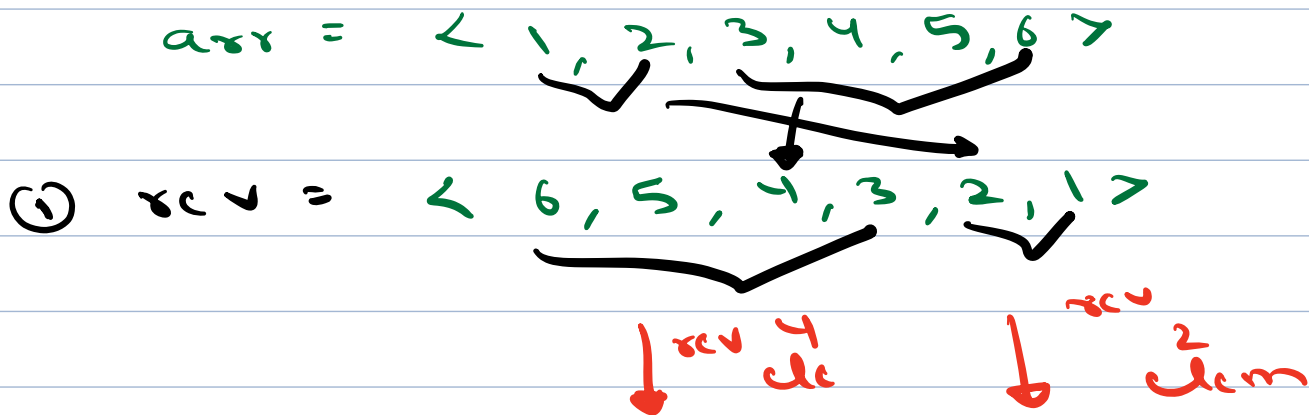
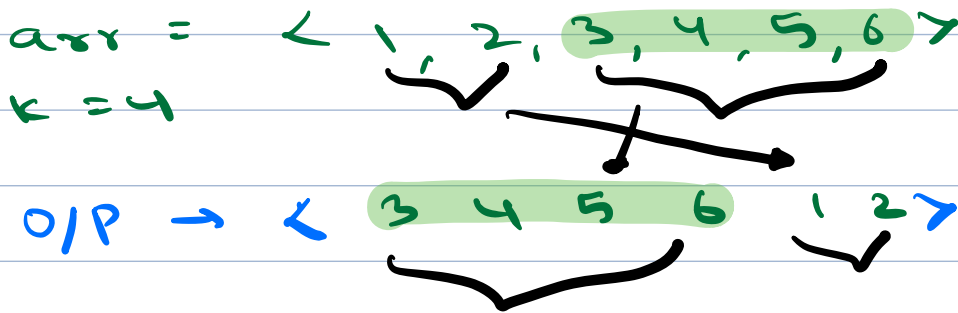
SC: $O(1)$

1 rotation $\rightarrow N-1$

K rotations $\rightarrow K \times (N-1)$
 $= KN$

$$1 \leq N, K \leq 10^5 \quad \text{iter} \rightarrow 10^{10} \text{ (TLE)}$$

Optimized $\rightarrow$



② O/P $\rightarrow \langle 3, 4, 5, 6, 1, 2 \rangle$

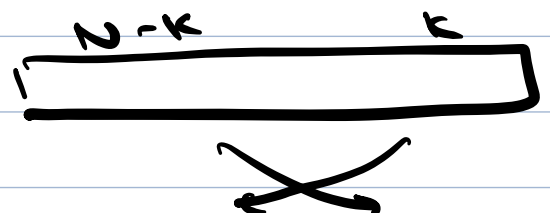
⑥ $K = K \% N$

① Reverse entire array reversePart(arr, N, 0, N-1)

② Reverse first K ele reversePart(arr, N, 0, K-1)

③ Reverse remaining $N-K$ ele reversePart(arr, N, K, N-1)

T.C : $O(3N)$
 $\Rightarrow O(N)$



SC: $O(1)$



$N = 100$

$K = 2000$

If $K > N$

$K-1 \rightarrow$ invalid idx

$N = 4$

	0	1	2	3	k		
$K = 0$	8	7	2	3	4	8	12
$K = 1$	3	8	7	2	5	9	13
$K = 2$	2	3	8	7	6	10	14
$K = 3$	7	2	3	8	7	11	15

$K = K \% N$

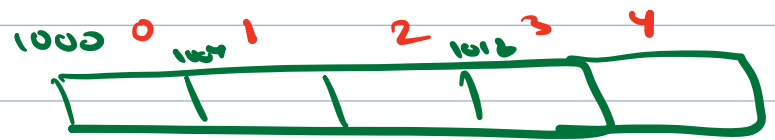


0 to $N-1$

Iterations $\rightarrow$ No. of times a loop runs

Why TC to access i^{th} element in array is $O(1)$! $a[i] \rightarrow O(1)$

a
1000



int a [5]

$$a[0] \rightarrow \begin{matrix} \text{addr} \\ 1000 \end{matrix}$$

$$a[1] \rightarrow 1000 + 4 = 1004$$

$$a[3] \rightarrow 1000 + 3 \times 4 = 1012$$

$$a[i] \rightarrow \begin{matrix} \text{starting} \\ \text{addr} \end{matrix} + 4i$$

$$(3) \quad 2^{10} \approx 10^3$$

$$1024 \approx 1000$$

$$\log_2 2^{10} \approx \log_2 10^3 = 10$$

$$\begin{aligned} \log_2 10^5 &= 5 \times \log_2 10 \\ &= 5 \times 3. \\ &= 15. \end{aligned}$$

$$\begin{aligned} &\log_a (b^c) \\ &= c \times \log_a b \end{aligned}$$

CPU $\rightarrow$ 1 GHz

$\downarrow$
 10^9 instructions/sec

Frequency

$\downarrow$
Hz

`print(Hi)  $\rightarrow$  ①`
`for (int  $i=0$  ;  $i < N$  ;  $i++$ ) <`
`$\rightarrow$  ①`
`print(Hi)  $\rightarrow$  ①`
`int a = 5  $\rightarrow$  ②`
`print(Bye)  $\rightarrow$  ①`

$2 + 10N$

$$10N = 10^9$$

$$N = \frac{10^9}{10} = 10^8$$

1 iter $\rightarrow$ 10 instructions
1 iter $\rightarrow$ 100 instructions

$$100N = 10^9$$

$$N = 10^7$$

Java

```
ArrayList<String> a = new ArrayList<>()  
a.add("50") // 50 is inserted at end  
a.clear()  
for (int i=0 ; i<a.size(); i++) <  
|_ System.out.print(a.get(i) + " ");  
|_
```

Python

```
a = []  
a.append("Orange") // Added orange at end  
a.clear()  
for i in range(len(a)):  
    print(a[i])
```

C++

```
vector<int> a  
a.push_back(60)  
a.clear()  
for (int i=0 ; i<a.size(); i++)  
    cout << a[i]
```